

Project Report: Sam's AQI Predictor

By: Samreen Fatima

Repository: <https://github.com/samreenfatimamalik/AQI-Prediction-System>

Live Dashboard: <https://sam-aqi-predictor.streamlit.app/>

Hopsworx Project: my_aqi_predictor (EU West, Project ID 41108)

1. Project Overview

Pearls AQI Predictor is a 100% serverless, end-to-end machine learning system that forecasts Air Quality Index (AQI), specifically PM2.5 concentration, for the next three days across five major Pakistani cities: **Lahore, Karachi, Islamabad, Faisalabad, and Peshawar.**

The system was built to demonstrate a complete production-style MLOps pipeline without relying on any always-on backend server. Data collection, feature engineering, model training, and prediction serving all run on scheduled automation and read/write directly against a managed feature store, with the dashboard computing predictions live at request time.

Objectives

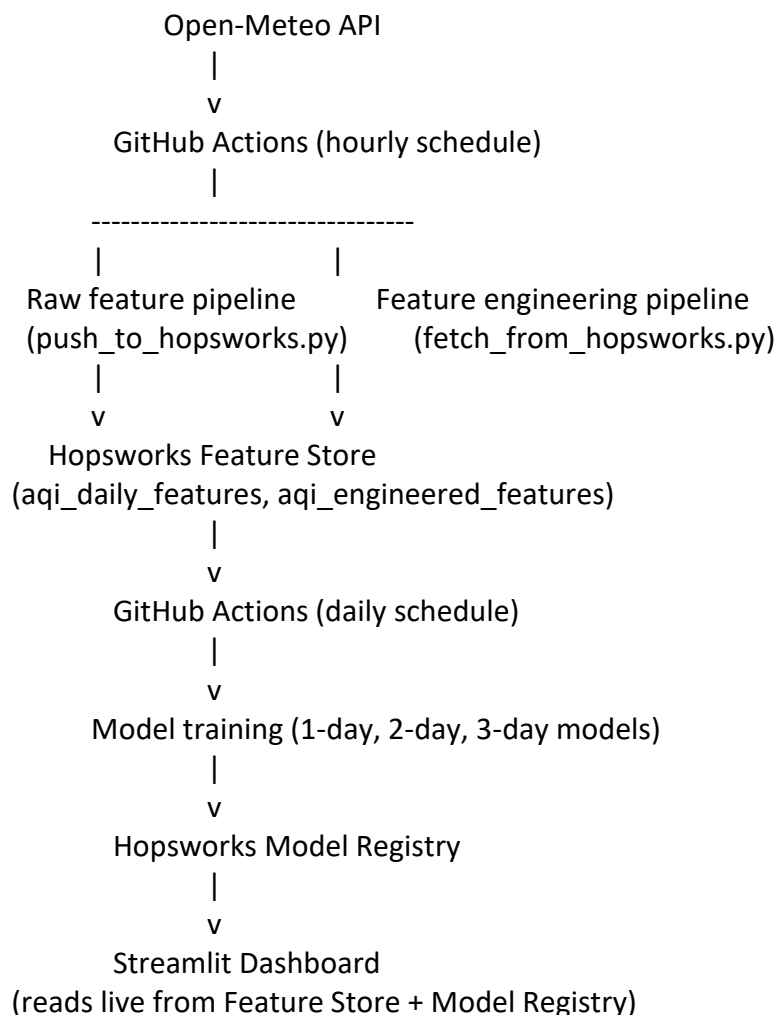
- Automatically collect weather and pollutant data on a schedule
 - Engineer meaningful time-series features (lags, rolling averages, calendar features)
 - Train and evaluate multiple regression models for 1-day, 2-day, and 3-day AQI forecasts
 - Store features and models in a centralized feature store and model registry
 - Serve live predictions through an interactive web dashboard
 - Explain predictions using SHAP feature importance
 - Alert users when forecasted AQI reaches hazardous levels
 - Automate the entire pipeline using CI/CD, with zero manual intervention after deployment
-

2. Technology Stack

Category	Tools Used
Language	Python
Data Source	Open-Meteo Air Quality API and Open-Meteo Weather API (no API key required)

Category	Tools Used
Feature Store & Model Registry	Hopsworks
Machine Learning	Scikit-learn — Linear Regression, Random Forest, HistGradientBoosting
Explainability	SHAP
Automation / CI-CD	GitHub Actions
Dashboard	Streamlit (deployed on Streamlit Community Cloud)
Visualization	Plotly
Version Control	Git and GitHub

3. System Architecture



There is no dedicated backend server. The dashboard connects to Hopsworks directly at runtime, loads the latest engineered features and the latest registered models, and computes predictions on the fly whenever a user opens the app.

4. Implementation Details

4.1 Feature Pipeline

`push_to_hopsworks.py` fetches hourly weather and air quality data for all five cities from the Open-Meteo API, aggregates it into daily averages, and inserts it into the `aqi_daily_features` feature group in Hopsworks (HUDI format, primary key `["city", "date"]`).

To stay within free-tier compute limits, the script pushes only a recent rolling window (approximately 20 days / ~85 rows) rather than the full historical dataset. It checks the most recent date already stored for each city and fetches only the missing days, so a failed run is automatically backfilled by the next successful one.

4.2 Feature Engineering

`fetch_from_hopsworks.py` reads the raw daily data, sorts it by `[city, date]`, and computes:

- **Time-based features:** day of week, month
- **Lag features:** PM2.5 value 1, 3, and 7 days earlier
- **Rolling features:** 3-day and 7-day rolling averages of PM2.5
- **Prediction targets:** PM2.5 value 1, 2, and 3 days ahead

All lag and rolling calculations are grouped by city before any `.shift()` or `.rolling()` operation, to prevent one city's data from leaking into another's calculations. The most recent rows — which don't yet have target values since the future hasn't occurred — are deliberately kept in the feature group so the dashboard can use them for live inference.

The engineered features are pushed to a second feature group, `aqi_engineered_features`.

4.3 Historical Backfill

A one-time backfill script (`fetch_multicity_daily.py`) was run to populate historical data across all five cities, giving the training pipeline enough history to learn meaningful lag and rolling patterns before the scheduled hourly pipeline took over ongoing collection.

4.4 Training Pipeline

Three models were trained, one per forecasting horizon, using Feature Views (`aqi_fv_1d`, `aqi_fv_2d`, `aqi_fv_3d`) built on top of the engineered feature group:

- `aqi_model_1d` — Linear Regression
- `aqi_model_2d` — HistGradientBoosting
- `aqi_model_3d` — HistGradientBoosting

Each model uses a **time-based train/test split** (older data for training, newer data for testing) rather than a random split, since a random split would leak future information backward in time. Models are evaluated with MAE, RMSE, and R^2 , and the best-performing model per horizon is registered in the Hopsworks Model Registry.

4.5 CI/CD Automation

Two GitHub Actions workflows keep the system running with no manual intervention:

- **Hourly Feature Pipeline** (hourly_feature_pipeline.yml) — fetches raw data, pushes it, then engineers and pushes derived features.
- **Daily Training Pipeline** — retrains all three models daily and re-registers them.

Both use workflow_dispatch so they can also be triggered manually for testing. The Hopsworks API key is stored as a GitHub repository secret.

4.6 Dashboard

Built with Streamlit and styled as an environmental sensor console, the dashboard includes:

- A city selector for all five cities
- A live AQI reading with color-coded status (Good, Moderate, Unhealthy, Very Unhealthy, Hazardous)
- A 3-day forecast panel
- A hazardous AQI alert banner
- A historical trend chart comparing observed vs. forecasted PM2.5
- A city comparison view
- A SHAP-based "Why this prediction" explainability tab
- A manual refresh option

5. Model Performance

Horizon	Best Model	R^2 Score
1-day ahead	Linear Regression	0.65
2-day ahead	HistGradientBoosting	0.51
3-day ahead	HistGradientBoosting	0.50

Performance decreases with longer horizons, which is expected: the model has no access to actual future weather forecasts and relies only on lag and rolling features from past days. Diagnosis of the remaining error showed that Lahore and Faisalabad account for most of the 2-day/3-day error, driven by unpredictable pollution spikes

(crop burning, dust storms) that historical PM2.5 lags alone cannot anticipate without real forecasted weather data as an input.

Feature engineering methodology (test additively, measure, revert): Additional engineered features (pm25_trend, pm25_rolling_max_7) were tested on the 3-day model but reduced R^2 from 0.499 to 0.484, so they were reverted for that horizon. The same features improved the 2-day model, so they were kept there. This selective, per-horizon approach reflects a measured rather than blanket feature-engineering strategy.

6. Problems Faced and Solutions

This project involved an extended, roughly two-week debugging period, most of which traced back to constraints of running production-style infrastructure on free-tier services rather than to application logic errors. Below is a chronological account of the major issues and how each was diagnosed and resolved.

6.1 Hopsworks Free-Tier Compute Cap Overage

Problem: Feature pipeline and training jobs began failing intermittently and unpredictably — jobs would die within seconds with no useful logs, making it look like a code or configuration bug.

Diagnosis: After extensive troubleshooting, contact with Hopsworks support (Lex) confirmed the root cause: the project had exceeded its free-tier compute allowance (~\$14 against a \$10/month cap), which caused the account to be silently paused. Jobs were being rejected instantly (roughly 8-second failures) with no execution and therefore no logs — a failure mode that is indistinguishable from a code bug without direct confirmation from the platform provider.

Solution: Support lifted the pause and waived the overage charge. Going forward, `push_to_hopsworks.py` was changed to push only a recent rolling window of data (~20 days / ~85 rows) instead of the full historical dataset, since full-history materialization was identified as the main driver of excess compute usage.

Ongoing risk: Full-history materialization will exceed the \$10 free cap again roughly two weeks after each monthly reset unless the underlying materialization job itself is changed to only process a rolling window of recent days (rather than the full table), or the project is moved to a paid plan. This is a known, tracked item for future work (see Section 8).

6.2 Misleading "AGGREGATING_LOGS" Stuck Status

Problem: A Hopsworks job appeared stuck indefinitely in an "AGGREGATING_LOGS" state in the UI, suggesting a hung or resource-consuming process.

Diagnosis: Investigation (with Hopsworks support) confirmed this was a **cosmetic display bug** — the underlying job was already dead and was not holding any compute resources. The UI simply failed to update its displayed status.

Solution: No action was needed on the job itself; this was documented as a known false signal so it wouldn't trigger unnecessary panic or unnecessary re-triggers in future debugging sessions.

6.3 Dashboard Runtime Errors (Local Testing)

Before deployment, the dashboard failed locally with three separate, cascading bugs that had to be diagnosed one at a time:

1. **Duplicate function definition:** dashboard.py contained two definitions of `load_all_features()`. Python silently used the second (broken) definition, overriding the working one, with no warning or error at import time.
2. **Missing lag feature computation in the fallback path:** A fallback data-loading path did not call `compute_lag_features()`, causing a `KeyError: 'pm25_lag_1'` whenever that path was triggered.
3. **Missing calendar features in engineered feature construction:** `build_engineered_features()` did not compute `day_of_week` and `month`, which downstream code expected to already exist.
4. **Duplicate dummy columns:** When the feature group already contained pre-engineered `city_*` one-hot dummy columns, the dashboard was re-generating them, producing duplicate columns and breaking model input alignment.

Solution: Each bug was isolated and fixed individually — removing the duplicate function, adding the missing `compute_lag_features()` call to the fallback path, adding calendar feature computation, and adding a check to skip dummy-column generation when those columns already exist in the source data. The dashboard and SHAP explainability were confirmed working locally only after all four fixes were applied together.

6.4 Streamlit Cloud Deployment: requirements.txt Parsing Error

Problem: After connecting the repo to Streamlit Community Cloud, the very first deploy attempt failed immediately during dependency installation with:

```
error: Couldn't parse requirement in `requirements.txt` at position 0
Caused by: Expected one of `@`, `(`, `<`, `=`, `>`, `~`, `!`, `;`, found ``
```

Diagnosis: The first line of `requirements.txt` contained a trailing comma (`streamlit,`), which is invalid syntax for a pip requirements file — packages must be listed one per line, not comma-separated.

Solution: Rewrote `requirements.txt` with one package per line and no commas, then pushed the fix. Streamlit Cloud auto-redeployed on the next push.

6.5 Streamlit Cloud Deployment: confluent-kafka Build Failure

Problem: After fixing the syntax error, the deploy failed again during a later attempt to build confluent-kafka from source:

fatal error: librdkafka/rdkafka.h: No such file or directory

Diagnosis: Streamlit Cloud's fast installer (uv) attempted to compile confluent-kafka from source and required a system-level C library (librdkafka) that isn't present in the deployment container. confluent-kafka had been included in requirements.txt (originally copied into the local setup instructions) but was never actually used anywhere in the project's code — this is an offline/batch pipeline using HUDI feature groups, not a real-time Kafka streaming pipeline.

Solution: Removed confluent-kafka from requirements.txt and from the README's local setup instructions entirely, since it was an unused dependency. This both fixed the build and reduced unnecessary install time and container size.

6.6 Streamlit Cloud App Sleep

Problem: The deployed dashboard periodically showed a "Zzzz — this app has gone to sleep due to inactivity" screen instead of the dashboard.

Diagnosis: This is expected, standard behavior for Streamlit Community Cloud's free tier — apps with no visitor traffic for an extended period are automatically put to sleep to conserve shared resources. It is unrelated to the Hopsworks compute cap issue and carries no cost risk.

Solution: The app can be manually woken via the "Yes, get this app back up!" button, which takes roughly 30–60 seconds. For demo purposes, the practice adopted was to visit the app shortly before any scheduled demo to ensure it's already awake.

6.7 Hopsworks Infrastructure Instability (General)

Beyond the compute cap issue, several other free-tier infrastructure quirks were identified and handled defensively in the pipeline code:

- **Arrow Flight outages** (grpc_status: 14 errors) during reads were resolved by falling back to the Hive read path (read_options={"use_hive": True}).
- **Stats computation failures:** Background statistics computation jobs occasionally caused HTTP 500 errors ("Transaction marked for rollback"). This was resolved by disabling statistics_config on the affected feature groups, since statistics weren't required for this use case.
- **Log expiry:** Free-tier job logs expire quickly, so logs had to be retrieved immediately after any failure to be useful for debugging; execution.download_logs() and dataset_api.download() sometimes returned empty results or HTTP 400 errors purely because logs had already been purged, not because of a permissions issue.

Solution pattern: Rather than trying to eliminate these transient failures entirely, the pipeline was designed to be resilient to them: automatic retries on reads/writes, gap-detection and backfill on every run, non-blocking writes that don't wait on slow background jobs, conflict avoidance (skipping a write if a prior background job is still in progress), and graceful degradation (exiting cleanly and logging the reason if Hopsworks is completely unreachable, so the next scheduled run can simply retry).

6.8 Data-Leakage Risk in Feature Engineering

Problem: Early feature engineering code risked mixing one city's historical data into another city's lag/rolling calculations, since Hopsworks does not guarantee row order on read.

Solution: Established a strict convention: always `sort_values(["city", "date"])` immediately after reading from Hopsworks, and always `groupby("city")` before applying any `.shift()` or `.rolling()` operation. This convention was applied consistently across the feature engineering and training scripts.

6.9 GitHub Actions Fresh-Checkout Environment

Problem: Scripts that assumed a local data/ directory already existed failed on GitHub Actions runners, since each run starts from a completely fresh repository checkout with no persisted local files.

Solution: Added `os.makedirs("data", exist_ok=True)` before any file write operation in scripts that run under GitHub Actions.

7. Key Learnings and Best Practices

- **Never use** `overwrite=True` on a HUDI feature group that has Feature Views attached — Hopsworks' dependency guard may or may not block this, and if it executes, it can wipe the entire table. Always use `plain insert()`, which behaves as an upsert.
- **Free-tier managed infrastructure fails differently than a bug does** — silent, fast, log-free job rejections are a strong signal to check account/billing status before assuming a code problem.
- **Push only what's needed.** Full-history writes to a feature store are expensive on free tiers; a rolling window is usually sufficient for both training and inference needs.
- **Group before you shift.** Any lag or rolling feature computation on multi-entity time series data must be grouped by entity (here, city) to avoid cross-contamination.
- **Test additively, measure, revert.** New engineered features should be evaluated per model/horizon rather than assumed universally beneficial — a feature that helps one forecast horizon can hurt another.

- **Free hosting platforms (Streamlit Cloud) enforce strict dependency syntax** and may lack system libraries needed to build certain packages from source; prefer packages with prebuilt wheels, and remove unused dependencies proactively rather than reactively.
 - **Design pipelines assuming imperfect infrastructure uptime.** Retry logic, gap detection, and graceful degradation matter more on free-tier managed services than on infrastructure you fully control.
-

8. Known Limitations

- Forecast accuracy decreases for the 2-day and 3-day horizons because the model has no access to actual forecasted weather for future dates, only past observations.
 - Sudden, event-driven pollution spikes (crop burning, dust storms) are difficult to predict without additional data sources such as satellite fire detection or dedicated weather forecast APIs.
 - The free tier of the feature store introduces occasional transient failures and a hard monthly compute cap, both of which the pipeline is designed to recover from and stay within, respectively, rather than eliminate entirely.
 - The `aqi_engineered_features` materialization job currently processes more history than strictly necessary; a rolling-window materialization fix is planned to keep the project comfortably within the free-tier compute budget indefinitely (see Section 9).
-

9. Future Improvements

- Change the `aqi_engineered_features` materialization job to process only a rolling window of recent days, to permanently avoid recurring compute cap overages
 - Incorporate actual weather forecast data (rather than only historical weather) to improve 2-day and 3-day accuracy
 - Add satellite-based fire and dust detection as an additional feature source
 - Experiment with deep learning models (LSTM or similar sequence models) for longer horizons
 - Add automated model comparison and rollback if a newly trained model performs worse than the currently deployed one
 - Extend coverage to additional cities
-

10. Conclusion

Pearls AQI Predictor demonstrates a complete, serverless MLOps pipeline for AQI forecasting — from automated data collection through feature engineering, model

training, and live prediction serving — built entirely on free-tier managed infrastructure. The majority of engineering effort in this project went into diagnosing and building resilience against the operational realities of free-tier cloud services (compute caps, transient outages, misleading status displays, and strict deployment constraints) rather than into the machine learning modeling itself, which is a realistic reflection of what production ML engineering work often looks like in practice.